📦 Supply Chain Operations & Automation Project

🚀 Overview

This project automates and analyzes Amazon-style supply chain operations by leveraging:

- **Gmail API**, **CSV ingestion**, and **Python preprocessing**
- **n8n.io** (Workflow automation)
- **Supabase** (PostgreSQL backend)
- **Quadratic** (Spreadsheet-like data analysis)

The pipeline helps monitor KPIs such as **OTIF %**, **Line Fill Rate**, **Volume Fill Rate**, and **Delivery Delays**, mimicking real-world warehouse and distribution metrics used at Amazon.

🧠 Key Features

✅ Automated Data Ingestion from different regions (US, Ahmedabad, Vadodara) via CSV files

✅ AI Workflow Automation using n8n.io to parse and insert data into Supabase DB

✅ Data Modeling with normalized tables for orders, products, and customers

✅ Business Insights generation with Quadratic to uncover delivery bottlenecks and revenue loss

✅ Supply Chain KPI Dashboards using industry-standard metrics

---

🛠️ Technologies Used

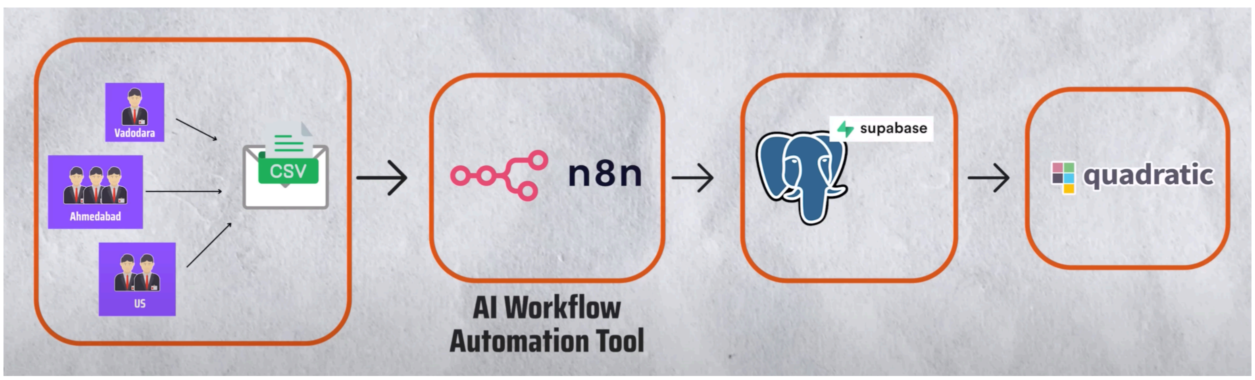
Tool	Purpose
n8n.io	Automation of Gmail → CSV → Supabase flow

Tool	Purpose
Supabase	PostgreSQL-based backend
Quadratic	Spreadsheet + Code for insight generation
Gmail API	Trigger for incoming order files
Python	Data cleaning and transformation

🖼️ Architecture Diagrams

🔄 End-to-End Supply Chain Automation Pipeline (CSV → n8n → Supabase → Quadratic)

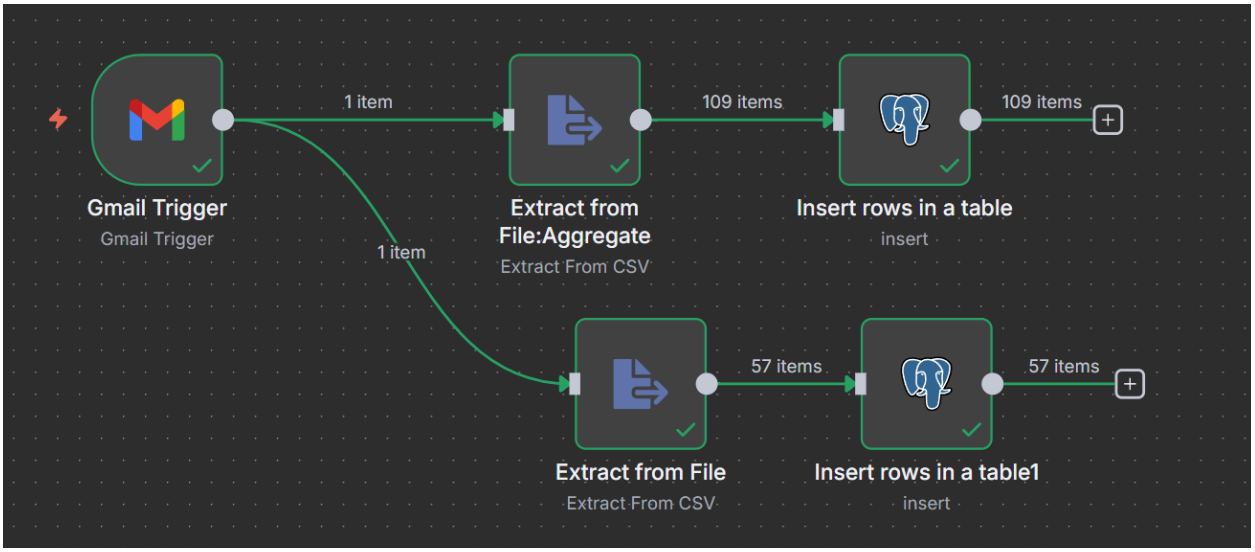
📊 Architecture Diagram



This diagram illustrates the flow from CSV ingestion to automation and analysis.

> CSV files from multiple regions (US, Ahmedabad, Vadodara) are automatically ingested, processed using n8n, and pushed to Supabase. Final analysis is done in Quadratic.

🔁 n8n Supply Chain Automation Workflow



> Workflow includes Gmail Trigger → CSV Extraction → Supabase Row Insertion. Data is routed dynamically based on file content.

📊 KPIs Tracked

- **On-Time Delivery % (OT)**
- **In-Full % (IF)**
- **OTIF %** (On-Time In-Full)
- **Line Fill Rate**
- **Volume Fill Rate**
- **Average Delay in Delivery**
- **Revenue Loss from Undelivered Orders**

📦 Supply Chain Automation Pipeline – Summary

In many industries, supply chain data still comes in through **emails** — vendors, suppliers, or internal teams send **Excel/CSV reports** to a designated inbox.

Manually downloading and processing these files is time-consuming, so the goal is to **automate the pipeline**.

🔷 Email Monitoring (n8n Integration)

- A workflow automation tool like **n8n** is configured to **monitor the inbox** for incoming vendor emails.
- Whenever a new **Excel/CSV file** arrives, n8n automatically picks it up.

🔷 Data Processing (n8n)

- n8n parses the Excel/CSV files.
- Performs **cleaning, formatting, and transformation** (e.g., standardizing column names, validating quantities, removing duplicates).
- Applies **business logic** such as:
  - Vendor tagging
  - SKU validation
  - Time-stamping

🔷 Data Storage (Supabase/Postgres)

- The processed data is loaded into a **Postgres database hosted on Supabase**.
- Ensures vendor supply chain data is **centralized, structured, and queryable**.

🔷 Analytics & Reporting (Quadratic)

- Data from Postgres is connected to **Quadratic** (spreadsheet-like analytics platform).
- Enables:
  - Data analysis
  - Dashboard creation

- Real-time insights for managers

🌍 Industry Use-Case Fit

- ✅ Reduces manual effort of downloading, merging, and cleaning Excel files
- ✅ Minimizes **human error** in data handling
- ✅ Provides **real-time supply chain visibility**
    - `Email → n8n → Supabase (Postgres) → Quadratic`
- ✅ Scales easily as more vendors or reports are added

🤖 Automated Data Migration using n8n (AI Agentic Workflow)

This workflow automates **data migration from Gmail → CSV → Postgres (Supabase)** using **n8n**.

Below are the setup steps:

🔑 Step 1: Sign Up for n8n

- Go to **n8n.io** and start a **14-day free trial**.
- Log in to the dashboard.
- Create a **new workflow**.

📩 Step 2: Monitor Gmail Inbox

1. Add the first node → **Gmail**.
2. Choose **Trigger: On Message Received**.
    - This means whenever a new email is received, the workflow is triggered.
3. Configure options:
    - **Mode:** Every minute

- **Uncheck:** Simplify
- **Add Filter:**
  - **Label:** `inbox`
  - **Subject:** `daily sales` (to match only specific patterns)

➡️ Now the workflow **monitors Gmail** for new CSV files in the inbox.

📁 Step 3: Extract Data from CSV

1. Add a **CSV Extract Node**.
2. In **Input Binary Field**, set:
   - `attachment_0` (this refers to the email attachment).
3. You can also view the extracted data in **JSON format** for validation.

➡️ This converts the incoming CSV attachment into a **JSON structure** that n8n can process.

🗄️ Step 4: Setup Supabase (Postgres DB Hosting)

Supabase is a cloud platform to host **Postgres** databases.

1. Go to Supabase → **Sign Up**.
2. Confirm email → Launch your project.
3. Create an **organization** and **project**.
4. Set a strong password (tip: can be same as account password).
5. Create the following **tables** in your project:
   - `dim_customers` → (dimension table: customer IDs, names, details)
   - `dim_products` → (dimension table: product IDs, categories, details)
   - `dim_targets_orders` → (dimension table: targets/planned orders)
   - `fact_order_line` → (fact table: transactional order data)

> 📌 Note:

> - **Fact table** = stores **transactional data** (e.g., sales, orders).
> - **Dimension tables** = store reference details (customers, products, etc.).

1. Go to **Connect → Connection String**:
    - Use **Session Pooler** for stable connection.
    - Copy details: **Host, Port, Database, User, Password**.

🔗 Step 5: Connect n8n to Supabase (Postgres)

1. In n8n canvas → Add a **Postgres Node**.
2. Enter the Supabase connection details (host, port, user, password, database).
3. Map JSON fields from CSV into the correct Postgres tables (`dim_*` or `fact_*`).

🔁 Step 6: Repeat for Other Files

- For each new file type (customers, products, targets, orders), repeat:
    - **Extract CSV → Convert JSON → Insert into Postgres**.

▶️ Step 7: Activate Workflow

1. Go back to **Workflow Overview** in n8n.
2. Toggle **Active**.
3. Your automation is now live:

📧 Gmail → 📁 CSV Extract → 🔄 JSON → 🗄️ Supabase (Postgres)

✅ Benefits

- No manual downloading or processing of emails.
- Real-time data ingestion from inbox to database.
- Clean ETL pipeline (Extract → Transform → Load).
- Scales across multiple vendors/files.

📊 Data Analysis in Quadratic (AI Spreadsheet)

Once the data pipeline is set up (Gmail → n8n → Supabase/Postgres), we can analyze it in **Quadratic**.

🔑 Step 1: Connect Quadratic to Postgres

1. Open **Quadratic** and create a **new file**.
2. Connect to your **Postgres database (Supabase)**:
    - **Host**: `<your_supabase_host>`
    - **Port**: `<port_number>`
    - **Database**: `<db_name>`
    - **User**: `<username>`
    - **Password**: `<password>`

📄 Step 2: Create Sheets for Each Table

- Create sheets in Quadratic with the following names:
    - `dim_customers`
    - `dim_products`
    - `dim_targets_orders`
    - `fact_order_line`
    - `fact_orders_aggregate`
    - `dim_date`
    - `exchange_rate`
    - `fact_summary`

For each sheet:

1. Click on the sheet name (e.g., `dim_customers`).
2. Select **Quick Selected Query**.
3. Run it → This fetches data directly from Postgres into Quadratic.

Repeat for all listed tables.

📅 Step 3: Create a Date Table (Prompt 1)

In the `dim_date` sheet, run the following AI prompt:

> Prompt:
>
> Create a date table that has the dates from **03-01-2025 (March 1st, 2025)** to **05-31-2025 (May 31st, 2025)**.

💱 Step 4: Create Exchange Rate Table (Prompt 2)

In the `exchange_rate` sheet, run:

> Prompt:
>
> Create an `exchange_rate` table for the period from **March 1st, 2025, to May 17th, 2025**.
>
> Use the following API request to fetch historical exchange rates:

```
response = requests.get(
    f'<https://openexchangerates.org/api/historical/2024-10-21.json?app_id=><
Replace your app id here>&symbols=INR'
)
```

- Replace `<Replace your app id here>` with your **Open Exchange Rates API Key**.
- Replace the **date** in the URL dynamically for each date in the range.

🧹 Step 5: Create a Fact Summary Table (Prompt 3)

In the `fact_summary` sheet, run:

> Prompt:
>
> Create a Python code that:

1. **Load data from:**
    - `fact_order_line` table
    - `dim_products` sheet (skip first row, use second row as headers)
    - `dim_customers` sheet (skip first row, use second row as headers)
    - `exchange_rate` sheet (skip first row, use second row as headers)
2. **Clean the data:**
    - Convert `product_id` and `customer_id` to numeric
    - Strip whitespace from IDs
    - Remove rows with NULL IDs
    - Convert IDs to integers
    - Convert dates to datetime
3. **Merge the tables:**
    - Orders with products → using `product_id`
    - Result with customers → using `customer_id`
    - Result with exchange rates → using `order_placement_date`
4. **Calculate total amounts:**
    - For **USD currency**: `price_USD * USD_INR_Rate * order_qty`
    - For **INR currency**: `price_INR * order_qty`
5. **Clean final output:**
    - Drop intermediate columns (`price_USD`, `price_INR`, `currency`, `Date`, `USD_INR_Rate`)
    - Keep only essential columns:
        - Order details
        - IDs

- Dates
- Quantities
- Delivery info
- `total_amount (in INR)`

✅ Final Workflow Recap

Supabase (Postgres) → Quadratic Sheets
↳ dim_customers

↳ dim_products

↳ dim_targets_orders

↳ fact_order_line

↳ fact_orders_aggregate

↳ dim_date (AI-generated)

↳ exchange_rate (API-based)

↳ fact_summary (clean + merged + calculated)

📊 Supply Chain KPI Creation & Analysis in Quadratic

✅ Step 1: Data Validation

Before calculating KPIs, validate that the data loaded in **Quadratic** matches what is stored in **Supabase**.

- Example: `dim_customers` table should have **35 rows** in both Supabase and Quadratic.
- Perform the same check for:
    - `dim_products`
    - `dim_targets_orders`
    - `fact_order_line`

- `fact_orders_aggregate`
- `dim_date`
- `exchange_rate`
- `fact_summary`

➡️ Confirm that row counts and data values match correctly.

Only after validation should we proceed to KPI calculation.

📈 Step 2: Business KPIs (Prompt 4)

Prompt to use in Quadratic:

> Prompt 4:
>
> Create the following KPIs
>
> 1. Total Order Lines
> 2. Line Fill Rate
> 3. Volume Fill Rate
> 4. Total Orders
> 5. On Time Delivery %
> 6. In Full Delivery %
> 7. On Time In Full %

📌 KPI Explanations in Easy Language

1. **Orders & Order Lines**

- **Order** = a unique request placed by a customer on a given date.
- **Order Line** = each individual product inside that order.
    - Example: If you order 4 notebooks and 2 pens on Amazon →
        - **Order ID** = 1 (one order)

- **Order Lines** = Notebook (line 1), Pen (line 2).

---

2. **Total Order Lines**

- Count of all the line items requested by customers across all orders.

---

3. **Line Fill Rate**

- Measures how many order lines were fulfilled vs. requested.
- Formula:

Line Fill Rate = (Order Lines Fulfilled ÷ Total Order Lines) × 100

- Example: Ordered 2 lines (Notebook, Pen).
- Fulfilled Notebook, failed Pen → 1/2 = **50%**.

---

4. **Volume Fill Rate**

- Measures how much quantity was shipped vs. requested.
- Formula:

Volume Fill Rate = (Total Quantity Shipped ÷ Total Quantity Ordered) × 100

- Example: Ordered 6 items (4 notebooks, 2 pens).
- Received 5 items (4 notebooks, 1 pen).
- 5/6 = **83%**.

---

5. **Total Orders**

- Count of all unique orders (unique order IDs).

---

6. **On Time Delivery %**

- At the **order level**, checks if an order was delivered **within the promised time**.
- An order is On Time **only if all items in that order arrive on time**.
- Important for warehouse & distribution teams.

7. In Full Delivery %

- At the **order level**, checks if the order was delivered **completely** as per the requested quantity.
- Important for supply planning teams.

8. On Time In Full (OTIF) %

- The strictest metric → checks if an order was delivered **both On Time & In Full**.
- An order is OTIF only if **all items are delivered in full and on time**.
- Important for **all supply chain teams** → reflects customer satisfaction and reliability.

🏆 Step 3: Top Customers (Prompt 5)

Prompt to use in Quadratic:

> Prompt 5:
>
> Show me **top 5 customers** based on **order value** and their **OTIF %, IF %, OT %**.
>
> Also add:
>
> - Customer Name
> - Customer ID
> - City

🇮🇳 Step 4: Top Customers (India) (Prompt 6)

Prompt to use in Quadratic:

> Prompt 6:
>
> Show me **top 5 customers in India** based on **order value** and their **OTIF %, IF %, OT %**.
>
> Also add:

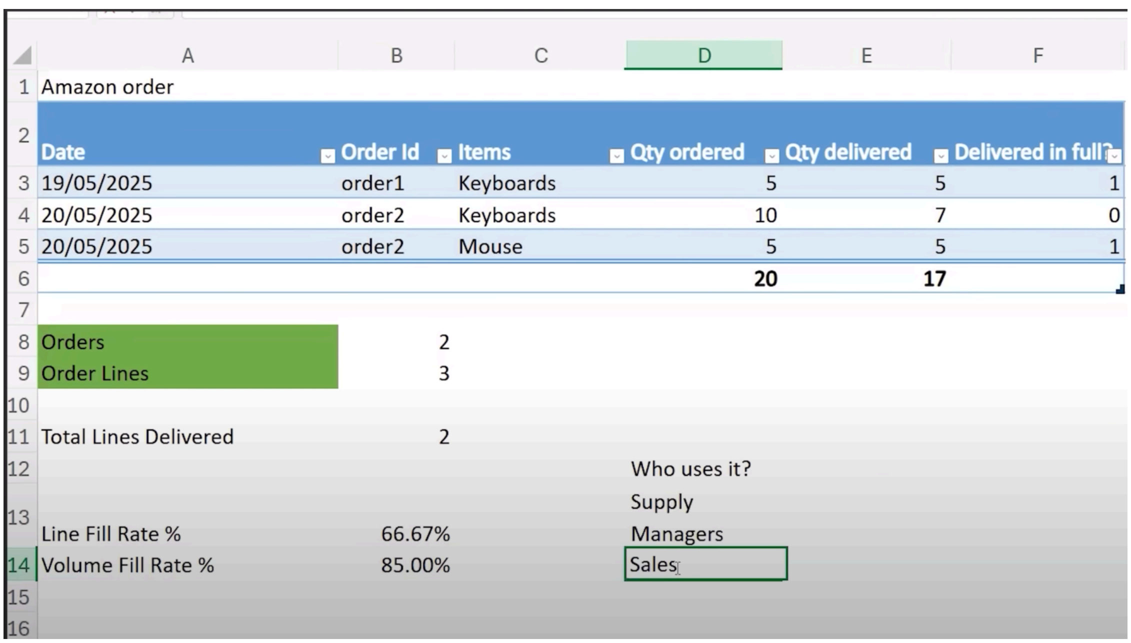
> - Customer Name
> - Customer ID
> - City

✅ Workflow Recap

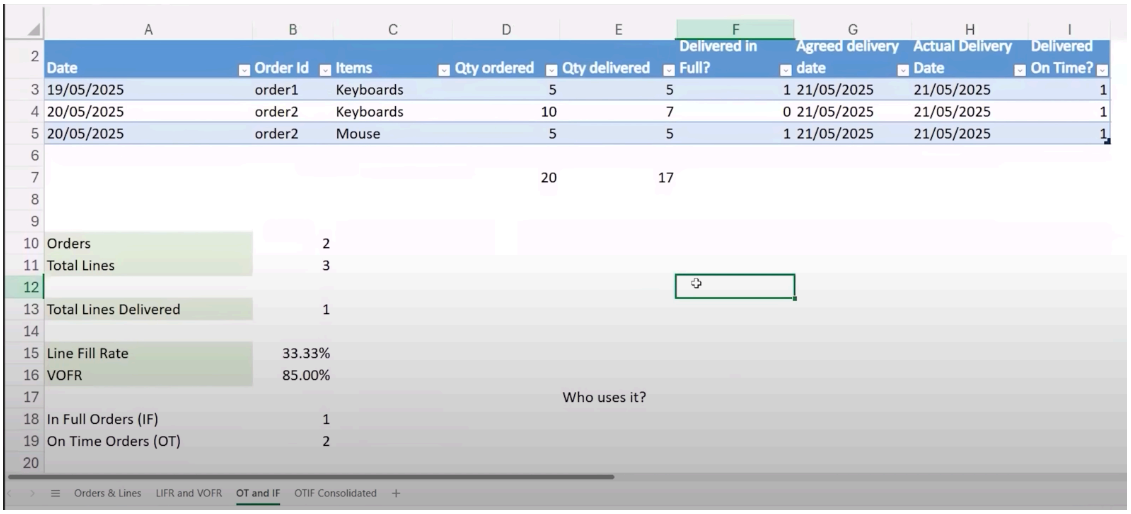
Data Validation (Supabase vs Quadratic) → KPI Creation → Top Customer Analysis

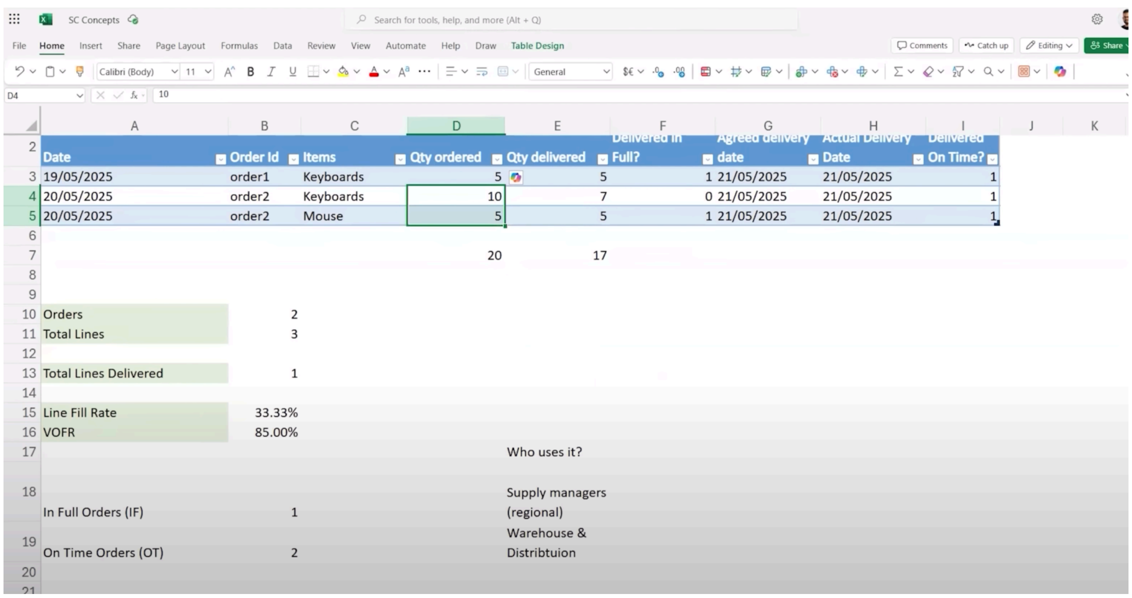
With these steps, you'll have:

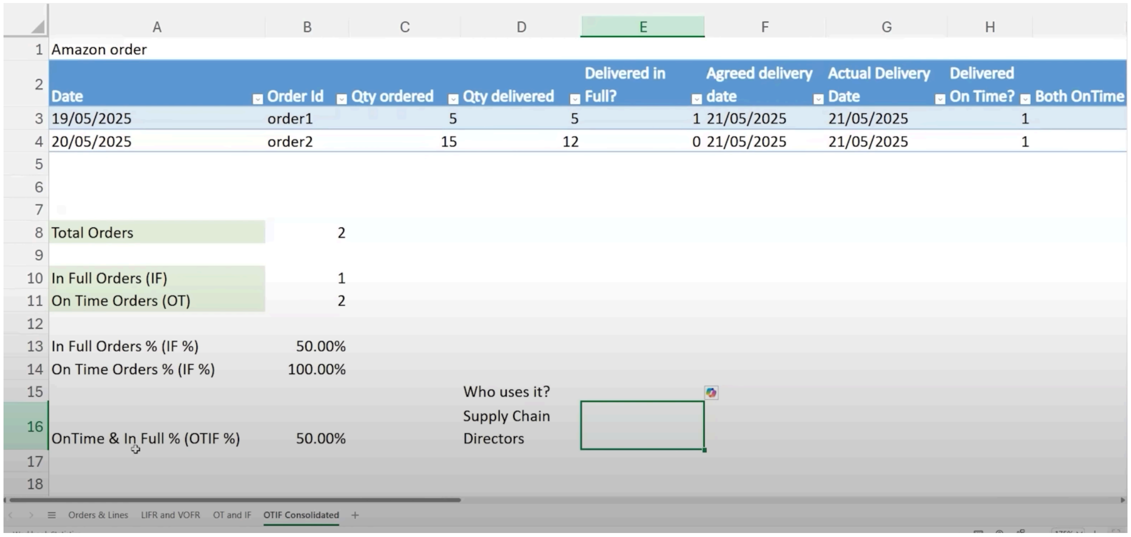
- Validated data consistency ✅
- Core supply chain KPIs ✅
- Top customer insights (global + India) ✅



	A	B	C	D	E	F
1	Amazon order					
2	Date	Order Id	Items	Qty ordered	Qty delivered	Delivered in full?
3	19/05/2025	order1	Keyboards	5	5	1
4	20/05/2025	order2	Keyboards	10	7	0
5	20/05/2025	order2	Mouse	5	5	1
6				**20**	**17**	
7						
8	Orders	2				
9	Order Lines	3				
10						
11	Total Lines Delivered	2				
12				Who uses it?		
13	Line Fill Rate %	66.67%		Supply Managers		
14	Volume Fill Rate %	85.00%		Sales		
15						
16						


Date | Order Id | Items | Qty ordered | Qty delivered | Delivered in Full? | Agreed delivery date | Actual Delivery Date | Delivered On Time?
19/05/2025 | order1 | Keyboards | 5 | 5 | 1 | 21/05/2025 | 21/05/2025 | 1
20/05/2025 | order2 | Keyboards | 10 | 7 | 0 | 21/05/2025 | 21/05/2025 | 1
20/05/2025 | order2 | Mouse | 5 | 5 | 1 | 21/05/2025 | 21/05/2025 | 1
20 | 17
Orders 2
Total Lines 3
Total Lines Delivered 1
Line Fill Rate 33.33%
VOFR 85.00%
Who uses it?
In Full Orders (IF) 1
On Time Orders (OT) 2
Orders & Lines
LIFR and VOFR
OT and IF
OTIF Consolidated

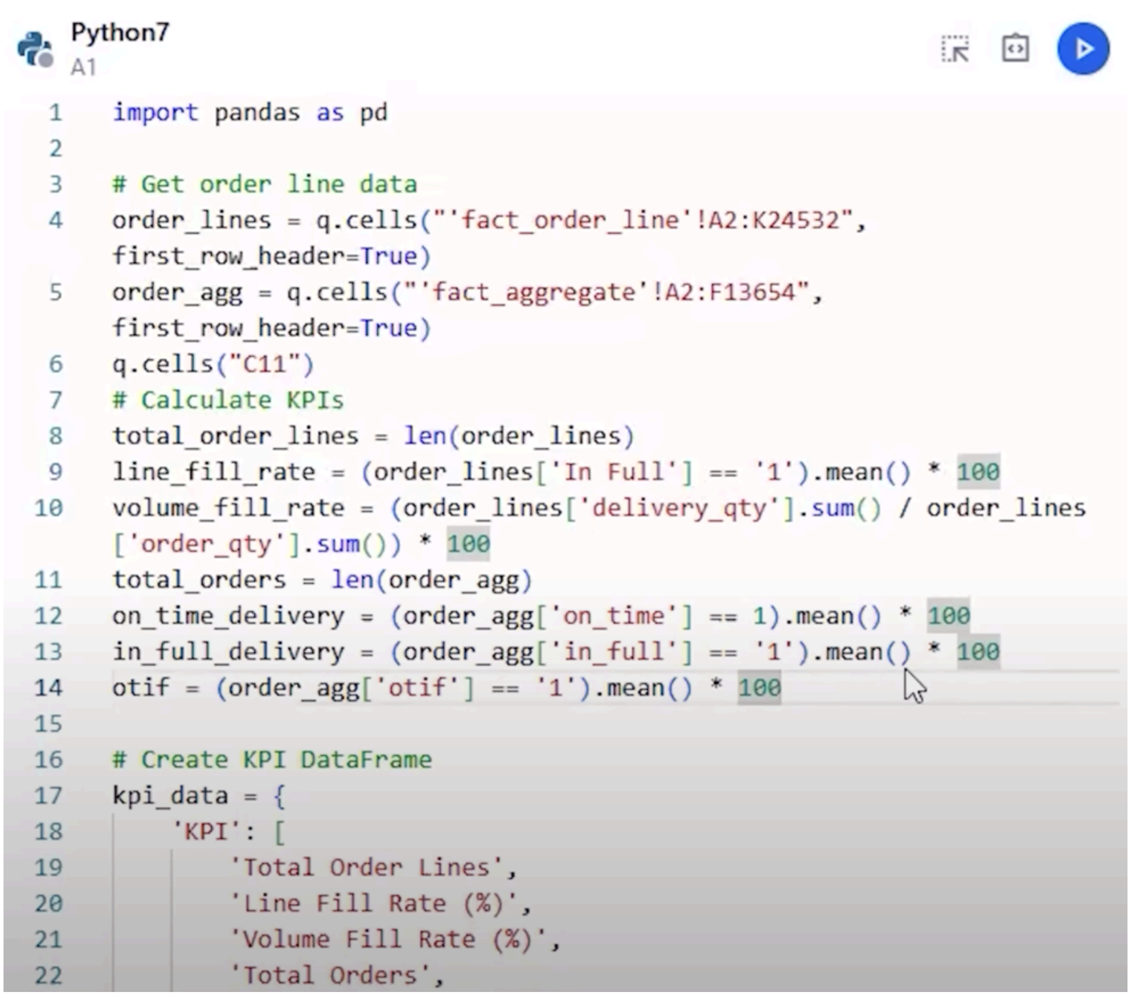

SC Concepts
Search for tools, help, and more (Alt + Q)
File
Home
Insert
Share
Page Layout
Formulas
Data
Review
View
Automate
Help
Draw
Table Design
Comments
Catch up
Editing
Share
Calibri (Body)
11
General
D4
10
Date | Order Id | Items | Qty ordered | Qty delivered | Delivered in Full? | Agreed delivery date | Actual Delivery Date | Delivered On Time?
19/05/2025 | order1 | Keyboards | 5 | 5 | 1 | 21/05/2025 | 21/05/2025 | 1
20/05/2025 | order2 | Keyboards | 10 | 7 | 0 | 21/05/2025 | 21/05/2025 | 1
20/05/2025 | order2 | Mouse | 5 | 5 | 1 | 21/05/2025 | 21/05/2025 | 1
20 | 17
Orders 2
Total Lines 3
Total Lines Delivered 1
Line Fill Rate 33.33%
VOFR 85.00%
Who uses it?
In Full Orders (IF) 1
Supply managers (regional)
On Time Orders (OT) 2
Warehouse & Distribtuion



Amazon order

Date	Order Id	Qty ordered	Qty delivered	Delivered in Full?	Agreed delivery date	Actual Delivery Date	Delivered On Time?	Both OnTime
19/05/2025	order1	5	5	1	21/05/2025	21/05/2025	1	
20/05/2025	order2	15	12	0	21/05/2025	21/05/2025	1	

Total Orders	2		
In Full Orders (IF)	1		
On Time Orders (OT)	2		
In Full Orders % (IF %)	50.00%		
On Time Orders % (IF %)	100.00%		
		Who uses it?	
OnTime & In Full % (OTIF %)	50.00%	Supply Chain Directors	




```
import pandas as pd

# Get order line data
order_lines = q.cells("'fact_order_line'!A2:K24532",
first_row_header=True)
order_agg = q.cells("'fact_aggregate'!A2:F13654",
first_row_header=True)
q.cells("C11")
# Calculate KPIs
total_order_lines = len(order_lines)
line_fill_rate = (order_lines['In Full'] == '1').mean() * 100
volume_fill_rate = (order_lines['delivery_qty'].sum() / order_lines
['order_qty'].sum()) * 100
total_orders = len(order_agg)
on_time_delivery = (order_agg['on_time'] == 1).mean() * 100
in_full_delivery = (order_agg['in_full'] == '1').mean() * 100
otif = (order_agg['otif'] == '1').mean() * 100

# Create KPI DataFrame
kpi_data = {
    'KPI': [
        'Total Order Lines',
        'Line Fill Rate (%)',
        'Volume Fill Rate (%)',
        'Total Orders',
```

Creating_OAuthClientID_for_Gmail_API

Steps are given in pdf: 3_Creating_OAuthClientID_for_Gmail_API

Creating N8n_Setup_in_Local_Host

steps are given in pdf: 1_N8n_Setup_in_Local_Host